

Jackson JSON API

Jose Velazquez Saenz

Bellevue University

CSD420: Advanced Java Programming

Professor Darrell Payne

May 24, 2026

Jackson JSON API

A Java JSON API is used to read, write, create, and process JSON data inside Java programs. For this paper, I selected the JSON API, and I chose the Jackson library, also known as FasterXML Jackson. Jackson is an open-source Java library for converting Java objects to and from JSON. This process is often called serialization and deserialization. Jackson is useful because many modern applications, web services, mobile apps, and APIs exchange data using JSON.

Jackson is not one single file. It is a group of related modules that work together. The three most common JAR files used for JSON processing are jackson-core, jackson-annotations, and jackson-databind. The Jackson-core module provides the lower-level JSON streaming parser and generator. According to the official FasterXML GitHub page, jackson-core contains the streaming parser and generator concepts used by Jackson's data processor. The Jackson-annotations module supports annotations that can be added to Java classes to control how JSON is created or read. The Jackson-databind module is usually the easiest for beginners because it includes the ObjectMapper class, which can map JSON data to and from Java objects.

One of Jackson's main features is data binding. Data binding allows a programmer to create a regular Java class and have Jackson automatically convert it to JSON. For example, a Student class with fields such as name, id, and grade can be converted to JSON without manually building the JSON string. The same process can also work in reverse. A JSON string can be read and converted into a Student object. The Jackson ObjectMapper documentation explains that ObjectMapper can read and write JSON from both Java objects and a general-purpose tree model.

Jackson also supports a tree model through JsonNode. This is useful when the JSON structure is not known in advance. Instead of mapping the JSON directly into a Java class, the programmer can read it as a tree and access specific fields by name. This is helpful when working with API responses or JSON files that may change over time. Another option in Jackson is the streaming API, which is lower-level and

can be more efficient for large files. Instead of loading the whole JSON document into memory, the streaming API reads through the data piece by piece. This can be useful in larger business systems that process large amounts of JSON data.

Jackson has a long history in the Java community. Oracle's Java Magazine notes that Jackson was first formally released in May 2009. Since then, it has become one of the most widely used JSON libraries for Java. It is used in many projects because it is flexible, fast, and works well with common development tools such as Maven, Gradle, Spring Boot, and regular Java applications. It is also licensed under the Apache License 2.0, which makes it usable in many personal, educational, and business projects.

Jackson supports several processes. First, it supports serialization, which converts Java objects to JSON. Second, it supports deserialization, which converts JSON back into Java objects. It supports configuration through annotations, such as changing property names or ignoring fields. It also supports reading and writing JSON from strings, files, streams, and web responses. Finally, it supports more advanced object features, including polymorphism and object identity, depending on the configuration. The Jackson-databind GitHub page describes the module as providing general-purpose data binding and a tree model that builds on the streaming API and uses Jackson annotations for configuration.

The main download location for Jackson JAR files is Maven Central. The official Jackson databind download page states that releases are available through the Maven Central repository. For a Java project, the needed JAR files are usually: jackson-core, jackson-annotations, and jackson-databind.

Jackson is a strong JSON API for Java because it offers many more features than other JSON libraries. Beginners can use ObjectMapper for simple object-to-object conversion, while more advanced programmers can use the tree model or the streaming API for greater control. It has been around for many years, has official documentation, and is available through Maven Central. Jackson is a good choice for JSON processing in Java.

References

Chandna, L. (2023, March 30). *Unlocking the power of fasterxml jackson library: A comprehensive guide*.

Medium. <https://medium.com/@lavneesh.chandna/unlocking-the-power-of-fasterxml-jackson-library-a-comprehensive-guide-3663b15781b>

Evans, B. (2023, January 2). *Efficient JSON serialization with Jackson and Java*. javamagazine.

<https://blogs.oracle.com/javamagazine/java-json-serialization-jackson/>

FasterXML/jackson-core. GitHub. (2026). <https://github.com/fasterxml/jackson-core>

FasterXML/jackson-databind. GitHub. (2026b). <https://github.com/FasterXML/jackson-databind>

Downloads. GitHub. (2026a). <https://github.com/FasterXML/jackson-databind/wiki/Downloads>